



Les devices pour les nuls

SDK Devices

[Un peu d'histoire](#)

[Architecture applicative](#)

[Qu'est-ce qu'une device au sein de notre solution ?](#)

[Modeles d'interfaces](#)

[Communication](#)

[Device Conference](#)

[Sender / Receiver](#)

[Device Managers](#)

[Activation + comportement de la device conference](#)

Les dispositifs

[RTC Devices](#)

[Audio](#)

[Video](#)

[IHealth - bluetooth](#)

[Spiromètre ProEXE](#)

[Ophtalmologie](#)

[Visionix](#)

[Topcon](#)

[Image device](#)

L'idée de ce document est de vulgariser l'implémentation et la communication des différents dispositifs médicaux de la solution. L'aspect plus technique est disponible dans d'autres documents confluence, la liste vous sera proposée en annexe.

SDK Devices

Un peu d'histoire

Tout d'abord, un peu d'histoire : Tessan ou plutôt Pharma Express, avant de faire des cabines, a essayé plusieurs choses.. mais ce qui nous intéresse ici c'est la conception d'un kit (SDK) de

dispositifs.

L'idée était de mettre à disposition plusieurs SDK dédiés à la mise en place d'un service de téléconsultation, avec notamment l'abstraction de la notion de dispositif (device) via l'un de ces sdk. Faciliter l'intégration d'un device, en abstrayant la notion de device pour faciliter l'intégration de tout type de protocole de communication (USB, audio, bluetooth...).

Disposant donc de cet outil, l'entreprise a décidé de se lancer et de créer sa propre solution médicale, profitant donc de cet atout.

Architecture applicative

Qu'est-ce qu'un device au sein de notre solution ?

Sur le plan physique, un device ou dispositif médical en bon français, est situé dans la cabine/borne. Son implementation au sein de notre système diffère selon le protocole de communication de l'appareil, le flux de données sortant est envoyé au travers du Slave afin d'être normalisé par le SDK afin de transiter à travers une **websocket SignalR** nommée **device conference**, dans la majorité des cas en passant par le serveur puis vers l'application doctor, et vice versa.

À travers différents niveaux **d'abstraction**, le but du SDK va principalement être de définir techniquement la notion de dispositif médical, de schématiser son protocole de communication au sein de notre solution tout en normalisant ses données et son comportement à travers la device conference.

Modeles d'interfaces

Le SDK dessine l'association entre un utilisateur, la deviceConference associée, et les devices actives.

Le SDK référence tous les contrats d'interfaces représentant ces trois notions.

Communication

Device Conference

La device conference est donc une websocket SignalR permettant la communication d'un dispositif vers l'application docteur et du patient.

Le slave va alors normaliser les données du dispositif grâce au SDK puis les envoyer dans la **deviceConference** via des senders associés pour être interprétés à la sortie par des receiver.

Sender / Receiver

L'abstraction d'un Sender ou d'un Receiver est nécessaire afin de pouvoir envoyer et recevoir les données d'un dispositif. Chaque dispositif possède sa propre classe qui hérite d'un

ADeviceSender ou d'un **ADeviceReceiver**.

[Sender](#) → envoie les données.

[Receiver](#) → reçoit les données.

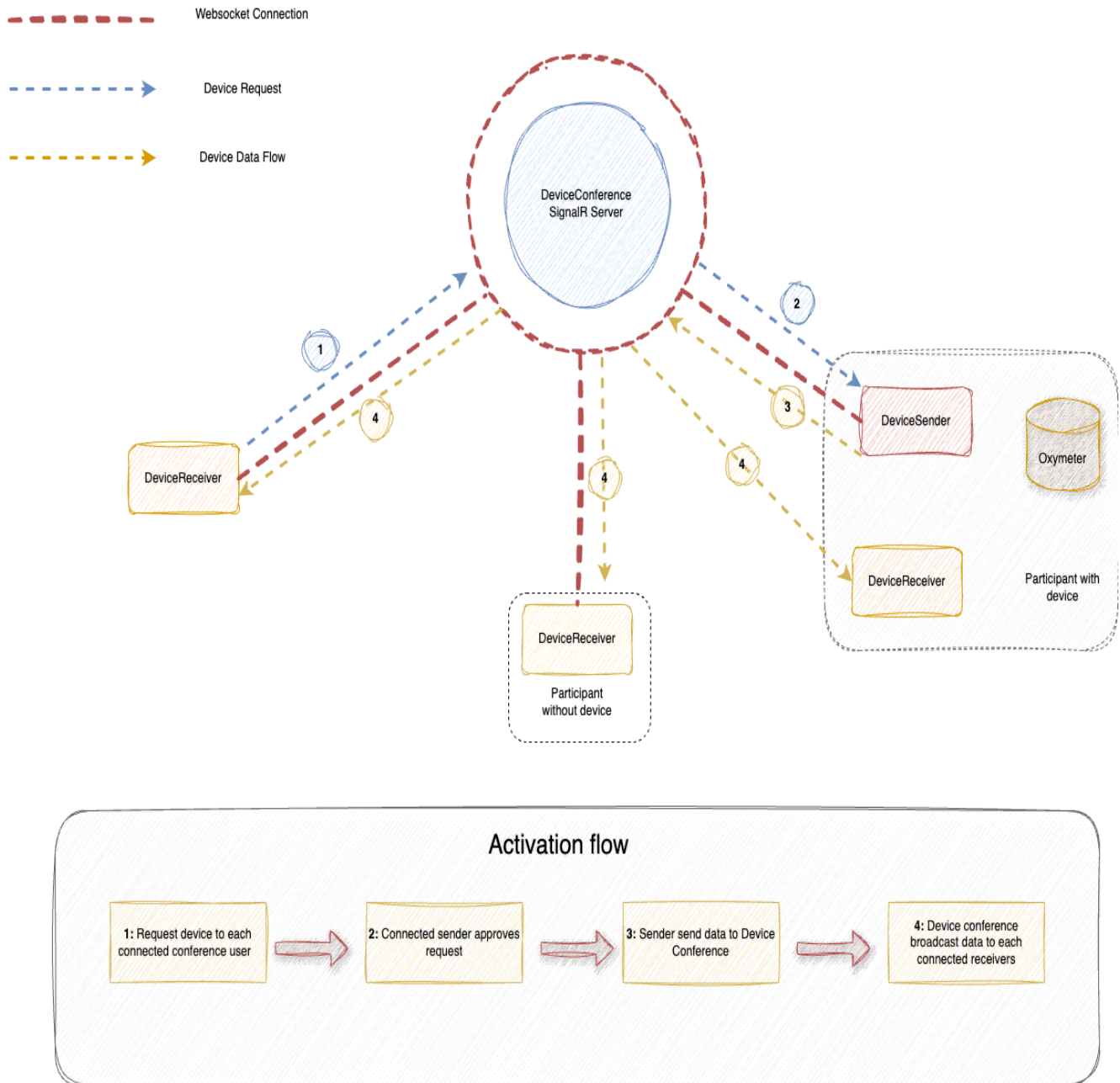
[Senders](#) [Receivers](#)

Device Managers

Chaque frontend implémente son propre service **device manager**, les classes finales représentant des dispositifs spécifiques sont créées dans les frontends.

Un device manager référence donc une map de senders et de receivers. Il pourra ainsi implémenter les fonctions essentielles représentant le cycle de vie d'une device au sein de la solution (start, stop...).

Activation + comportement de la device conference



Les dispositifs

RTC Devices

À ce jour, trois dispositifs de flux audio et vidéo sont branchés physiquement sur le NUC. Ces derniers utilisent deux abstractions supplémentaires qui sont

AStreamDevice[Sender/Receiver] héritant aussi d'une classe **RTCDevice[Sender/Receiver]**.

Ils utilisent alors le protocole **WebRTC** pour communiquer.

[RTC Communication Flow](#)

Audio

Le stethoscope, branché directement sur la prise jack du NUC, utilise un driver audio et fournit alors un stream audio qui sera donc envoyé via un **RTCDeviceSender**.

Video

Deux autres dispositifs, l'otoscope et le dermatoscope sont branchés directement sur le NUC, cette fois-ci en USB, ils fournissent un flux vidéo aussi transmis via un **RTCDeviceSender**.

IHealth - bluetooth

Tout les dispositifs **IHealth** sont en bluetooth, historiquement le NUC n'était pas équipé de ce moyen de communication, il a donc été nécessaire de les associés en bluetooth à la tablette (master).

IHealth fournit un SDK Java, ce qui a permis l'implémentation d'un service **device manager** afin de l'exploiter et donc d'utiliser les différents dispositifs via android. Le Master étant développé en Java/Kotlin.

Voici la liste des dispositifs IHealth :

- Oxymètre
- Tensiomètre
- Balance et balance à impédance
- Thermomètre

Côté **Sender** (slave), ces dispositifs sont des **DeviceBridged**. C'est à dire que les données transitent via le **bridge** (sur le NUC) pour communiquer du Master vers le Slave.

IHealth Communication Flow

Spiromètre ProEXE

Le spiromètre est lui aussi en bluetooth, mais vient d'un autre partenaire : ProEXE (anciennement AIOCaire).

Autre partenaire, implémentation différente.

Le dispositif fonctionne à l'aide d'une application mobile ProEXE disponible via le store de google. Faute d'api ou de SDK, celle-ci est alors instanciée "cachée" directement dans le Master Android, un event manager spécifique implémentant un protocole custom permet de communiquer entre cette application et le master.

La communication est ensuite similaire aux **DeviceBridged** ci-dessus.

Côté médecin, une application spiromètre (Tessan) est lancée dans une nouvelle fenêtre permettant le pilotage du dispositif à distance. Le démarrage, calibrage et prises de mesures peuvent alors avoir lieu via l'application.

 [AIOCare Communication Flow](#)

Ophtalmologie

Visionix

Il s'agit de l'ancienne table de télé-ophtalmologie, mise en place avec le partenaire Visionix (anciennement Luneau).

Cette table dispose de deux dispositifs:

- VX650
- EYE-REFRACT

Les flux des dispositifs transitent via une **Websocket** vers le slave, la représentation des devices Luneau au sein du SDK les oriente vers des RTCDevices (flux video).

Comme pour le spiromètre, le médecin dispose d'une application dédiée au dispositif qui se lance dans une nouvelle fenêtre au démarrage de celui-ci.

 [VISIONIX](#)  [Luneau Communication Flow](#)

Topcon

Il s'agit de la nouvelle table de télé-ophtalmologie, avec un nouveau partenaire : **Topcon**.

Cette table dispose de trois dispositifs :

- **CT1P**
- **Chronos**
- **NW500**

Ces derniers sont connectés sur le même réseau avec des adresses ip fixes. Chaque dispositif expose une application web accessible localement. Afin d'accéder à ces interfaces et donc d'exposer l'ip de ces serveurs web à notre solution, nous avons dû mettre en place un **proxy** (Ngrok) afin d'exposer ces applications à travers le cloud.

Lorsque le médecin souhaite utiliser un dispositif TOPCON, un tunnel Ngrok est alors instancié sur le Salve afin d'exposer l'url de l'interface du dispositif cible puis remonter cette url à travers le proxy via un Sender associé.

Ensuite une application s'ouvre sur le navigateur du médecin, avec une iframe invoquant l'adresse du proxy et permet donc d'accéder à l'interface du dispositif.

[📄 Topcon](#) [📄 Proxy Device Communication Flow \(TOPCON\)](#) [📄 Ngrok](#)

Image device

Il ne s'agit pas vraiment d'un dispositif médical, mais l'image device permet à un patient de partager des documents directement avec son smartphone.

Après avoir scanné un QR code sur le master, le téléphone ouvrira un navigateur pointant sur une application web Tessan.

Grâce au token récupéré au moment du scan, l'appareil se connecte à la deviceConference. Un Sender va être créé et offrir la possibilité d'uploader une image ou un document au travers de notre système.

[📄 Image Device Communication Flow](#)